墨韵 AI 文风优化 & 小说模仿功能计划

一、问题分析：AI味从哪来

当前系统的 prompt 模式是：

```
小说总纲 + 章节大纲 + 上下文 + "请生成第X章正文"
```

AI味的典型表现：

- 大量使用"然而""不禁""似乎""一种...的感觉""眼神中闪过一丝..."
- 段落结构雷同：动作→心理→对话→环境，机械循环
- 情绪描写公式化："心中涌起一股...""嘴角微微上扬"
- 对话过于工整，缺乏口语化/方言/语气词
- 缺乏作者个人习惯用语、独特比喻

根因： LLM 在没有任何风格约束时，会输出"最大概率"的文本，这就是"AI味"——最安全、最通用、最无聊的表达。

当前 prompt 结构

```
SystemPrompt: "你是专业的小说写作Agent，遵守以下铁律..."
UserPrompt: 总纲 + 章纲 + 人物状态 + 伏笔 + 上下文 + "请生成正文"
```

AI味的6个典型症状

症状	示例	根因
高频套话	"不禁""似乎""眼神中闪过一丝""嘴角微微上扬"	LLM 选最高概率词
结构雷同	动作→心理→对话→环境，机械循环	缺乏段落节奏约束
情绪公式化	"心中涌起一股...""一股暖流涌上心头"	没有情绪表达多样性要求
对话工整	每句都语法完整，没有口语/省略/语气词	缺乏人物语言风格设定
比喻泛滥	大量明喻"像...一样""如同..."	缺乏禁止滥用修辞的指令
缺乏"脏感"	文字过于干净，没有作者个人癖好	没有文风指纹注入

二、方案一：Anti-AI Prompt 工程（立即可用）

原理

在 SystemPrompt 中注入反 AI 套话的硬约束，同时要求输出多样性。

实现方式

修改 `WritingAgent.java` 的 SystemPrompt，新增以下模块：

2.1 反 AI 套话黑名单

```
【文风铁律——违反即扣分】
1. 禁止使用以下高频 AI 套话（全文出现即失败）：
   "不禁""似乎""仿佛""眼神中闪过""嘴角微微上扬""心中涌起一股"
   "一种...的感觉""空气中弥漫着""不约而同""若有所思"
   "意味深长""难以名状""若隐若现""一股暖流涌上心头"
   "与此同时""话音刚落""沉默片刻""微微一怔"

2. 禁止过度使用明喻（每千字不超过2个"像""如同""仿佛"）

3. 对话必须口语化：允许省略主语、语气词（"嗯""哈""切"）、
   打断（用破折号）、吞字、方言，禁止每句都语法完整

4. 心理描写不要"XX想道"，而是通过动作/细节暗示情绪
   ✗ "他心想：这下完了"
   ✓ "他盯着屏幕，手指停在键盘上，一个字也敲不出来"
```

2.2 段落节奏多样化

```
【段落节奏要求】
- 长短交替：至少30%的段落≤2行（50字以内）
- 禁止连续3段以上都是"一段描写+一段对话+一段心理"
- 战斗/紧张场景：短句为主，1-3句一段
- 日常/对话场景：可以长段落，但对话必须单独成段
```

2.3 文风参考片段注入

在 `ContextBuilderService.buildWritingContext()` 中，新增一个 `styleReference` 字段：

- 从该小说已有的前几章中，抽取最能代表作者风格的 2-3 段（500-800字）
- 注入到 prompt 中作为 `【文风参考——请模仿此风格写作】`

```
【文风参考——请模仿此风格写作】
以下片段来自本书前文，请模仿其句式、节奏、修辞习惯：

---
{从已有章节中抽取的代表性段落}
---

写作时参考上述片段的：
- 句子长短比例
- 隶落密度
- 对话风格
- 修辞偏好
```

文件改动

文件	改动
`WritingAgent.java`	SystemPrompt 新增反 AI 套话规则 + 段落节奏要求
`ContextBuilderService.java`	新增 `buildStyleReference()` 方法，从已有章节抽取文风片段
`StreamingChapterController.java`	userPrompt 中注入文风参考

工作量：1-2天

三、方案二：用户文风指纹系统（中期）

原理

允许用户手动设置自己的写作风格偏好，或从自己写过的文本中自动提取风格特征。

3.1 数据模型

新增表 `novel_style_profile`：

```
CREATE TABLE novel_style_profile (
  profile_id     VARCHAR(32) PRIMARY KEY,
  novel_id       VARCHAR(32) NOT NULL,

  -- 风格标签（用户手动选择 或 AI提取）
  tone           VARCHAR(50),     -- 叙事基调：轻松/热血/沉重/暗黑/温馨
  pov            VARCHAR(20),     -- 视角：第一人称/第三人称限知/第三人称全知
  tense          VARCHAR(10),     -- 时态：过去/现在

  -- 语言特征（JSON存储）
  vocab_style    JSON,            -- {"常用词":["..."],"禁用词":["..."],"语气
词":"..."}
  sentence_pattern JSON,          -- {"平均句长":25,"短句占比":0.3,"喜欢用反问":true}

  -- 示例文本（用于 few-shot）
  sample_texts   TEXT,            -- 用户粘贴自己写的代表段落（3-5段）

  -- 自由指令
  custom_prompt TEXT,             -- 用户自定义写作要求

  create_time    DATETIME,
  update_time    DATETIME
);
```

3.2 风格提取流程

```
用户粘贴/上传一段文本（500-2000字）
  ↓
LLM 分析（STRUCTURED 模式，低温度）：
  提取以下特征 → JSON
  - 叙事视角
  - 句子长度分布
  - 高频词汇（作者癖好用词）
  - 段落节奏特点
  - 对话风格
  - 修辞偏好
  - 情绪表达方式
  ↓
存入 novel_style_profile
  ↓
生成时注入到 WritingAgent 的 SystemPrompt 中
```

3.3 前端交互

在小说详情页新增「文风设置」面板：

- 选项卡1：基础设置（基调/视角/时态，下拉选择）
- 选项卡2：示例文本（粘贴代表段落）
- 选项卡3：自定义指令（自由文本框，如"我喜欢用短句，对话多用反问"）
- 选项卡4：风格分析结果（AI提取的特征，只读展示）

文件改动

层级	文件	改动
数据库	新建 `novel_style_profile` 表	DDL
后端	`StyleProfileService.java`（新建）	CRUD + AI提取
后端	`StyleProfileController.java`（新建）	REST API
后端	`WritingAgent.java`	SystemPrompt 动态注入风格
后端	`ContextBuilderService.java`	构建风格上下文
前端	`StylePanel.vue`（新建）	文风设置面板
前端	`NovelDetail.vue`	引入 StylePanel

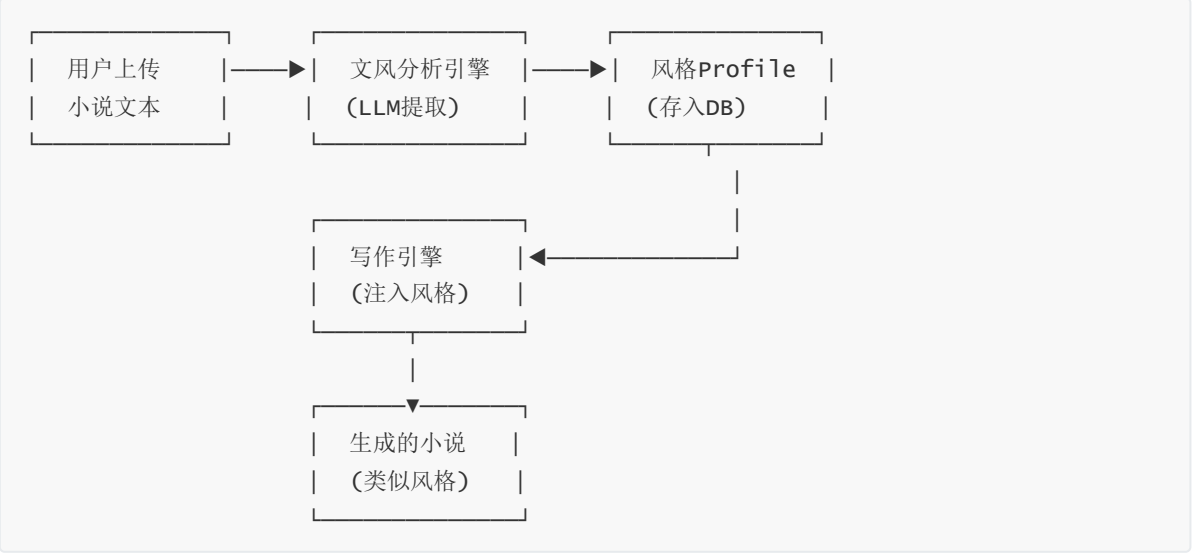
工作量：3-5天

四、方案三：小说模仿功能（导入→模仿→生成）

4.1 用户故事

> 用户有一本喜欢的小说（如《诡秘之主》），想写一本风格类似的小说。
> 用户导入《诡秘之主》的文本，系统分析其文风，然后用这个文风来写用户自己的小说。

4.2 架构设计



4.3 实现步骤

Step 1: 文本导入（前端+后端）

前端：

- 新建 `ImportNovel.vue` 页面
- 支持3种导入方式：
  1. 粘贴文本（最简单，适合短文本）
  2. 上传 .txt 文件（支持大文件，分块上传）
  3. 输入网页URL（可选，后期实现）
- 导入后预览前1000字，确认内容正确

后端：

- 新建 `ImportController.java`
- `POST /import/text` — 接收文本内容
- `POST /import/file` — 接收文件上传
- 文本分段处理：按章节自动分割（正则匹配 "第X章"）
- 存入 MongoDB `imported_novel` 集合

Step 2: 文风深度分析（后端核心）

分析维度（LLM 分析，分多次调用）：

```
{
  "宏观特征": {
    "题材": "玄幻",
    "叙事视角": "第一人称",
    "时态": "过去时",
    "基调": "热血中带幽默",
    "整体节奏": "前慢后快"
  },
  "语言指纹": {
    "高频词汇": ["念头通达","嘴角噙着笑","眼底深处"],
    "禁用词": ["不禁","似乎","仿佛"],
    "句式特征": "短句为主，长句用于描述战斗",
    "段落密度": "平均每段3-5行",
    "对话特色": "角色各有口头禅，常用反问"
  },
  "修辞偏好": {
    "比喻类型": "喜欢用战斗/武学相关的比喻",
    "排比": "少用",
    "反问": "高频使用"
  },
  "情绪表达": {
    "愤怒": "通过动作（握拳、青筋暴起）而非心理描写",
    "喜悦": "通过对话和外部反应",
    "紧张": "环境描写+短句"
  },
  "段落样本": [
    {
      "类型": "战斗场景",
      "原文": "...(500字代表性片段)..."
    },
    {
      "类型": "日常对话",
      "原文": "...(500字代表性片段)..."
    }
  ]
}
```

实现方式：

```
// StyleAnalyzerService.java（新建）

@Service
public class StyleAnalyzerService {

    /**
     * 分析导入文本的文风
     * 分3步调用LLM：
     * 1. 宏观特征分析（总览全文）
     * 2. 语言指纹提取（抽样分析前3章+中间3章+后3章）
     * 3. 段落样本提取（挑选代表性段落）
     */
```

```
    public StyleProfile analyze(String importedNovelId) {
        // 获取导入的文本
        List<ImportedChapter> chapters =
importedChapterMapper.selectByNovelId(importedNovelId);

        // 采样：取前3章 + 中间3章 + 后3章
        List<ImportedChapter> samples = sampleChapters(chapters, 9);

        // Step 1: 宏观特征
        String macroPrompt = buildMacroPrompt(samples);
        MacroAnalysis macro = llmAnalyze(macroPrompt, MacroAnalysis.class);

        // Step 2: 语言指纹
        String fingerprintPrompt = buildFingerprintPrompt(samples);
        LanguageFingerprint fingerprint = llmAnalyze(fingerprintPrompt,
LanguageFingerprint.class);

        // Step 3: 段落样本
        String samplePrompt = buildSamplePrompt(chapters);
        List<StyleSample> styleSamples = llmAnalyze(samplePrompt, List.class);

        // 合并为 StyleProfile
        return StyleProfile.builder()
            .macro(macro)
            .fingerprint(fingerprint)
            .samples(styleSamples)
            .build();
    }
}
```

Step 3: 风格注入写作（核心修改）

修改 `WritingAgent.java` 的 SystemPrompt，动态注入风格：

```
【模仿风格要求——你正在模仿《{sourceNovelName}》的文风写作】

{StyleProfile的JSON摘要}

【段落参考——请模仿以下段落的句式、节奏、修辞】

战斗场景参考：
---
{从导入小说中提取的战斗段落}
---

对话场景参考：
---
{从导入小说中提取的对话段落}
---

【模仿要求】
1. 句式：模仿参考段落的句子长短比例和断句方式
2. 词汇：使用与参考段落相近的词汇风格，禁止使用截然不同的表达
3. 修辞：模仿参考段落的比喻类型和使用频率
4. 节奏：保持与参考段落一致的段落密度
```

```
5. 注意：是模仿风格，不是抄袭内容。情节必须100%原创
```

Step 4: 前端交互流程

```
用户流程：
1. 点击「导入小说模仿文风」
2. 粘贴或上传文本 → 预览确认
3. 点击「分析文风」→ 显示进度条（分析中...）
4. 分析完成 → 展示风格报告卡片：
   - 题材：玄幻 | 视角：第一人称 | 基调：热血
   - 高频词：念头通达、嘴角噙着笑...
   - 对话特色：角色有口头禅，常用反问
   - 3个风格示例段落
5. 用户确认 → 绑定到目标小说
6. 之后生成章节时自动应用该风格
```

4.4 数据模型

新增表/集合：

```
-- 导入小说记录
CREATE TABLE imported_novel (
  import_id     VARCHAR(32) PRIMARY KEY,
  user_id       VARCHAR(32),
  novel_name    VARCHAR(200),
  source_type   VARCHAR(20),   -- text/file/url
  total_chapters INT,
  total_words   BIGINT,
  status        INT,           -- 0=待分析, 1=分析中, 2=已完成
  create_time   DATETIME
);

-- 导入的章节（MongoDB）
-- collection: imported_chapter
-- { importId, chapterNum, title, content }

-- 风格Profile
CREATE TABLE style_profile (
  profile_id    VARCHAR(32) PRIMARY KEY,
  import_id     VARCHAR(32),          -- 关联导入小说
  target_novel_id VARCHAR(32),        -- 绑定到的目标小说

  -- 宏观特征
  genre         VARCHAR(50),
  pov           VARCHAR(20),
  tone          VARCHAR(50),

  -- 语言指纹（JSON）
  fingerprint   JSON,

  -- 段落样本（MongoDB）
  -- collection: style_sample

  -- 自由指令
```

```
  custom_prompt TEXT,

  status        INT,                    -- 0=待分析, 1=分析中, 2=已完成
  create_time   DATETIME,
  update_time   DATETIME
);
```

4.5 文件改动清单

层级	新建文件	修改文件
数据库	`imported_novel.sql`	—
后端	`ImportController.java`	—
后端	`ImportService.java`	—
后端	`StyleAnalyzerService.java`	—
后端	`StyleProfileController.java`	—
后端	`StyleProfileService.java`	—
后端	`ImportedNovel.java` (entity)	—
后端	`StyleProfile.java` (entity)	—
后端	`ImportedNovelMapper.java` + XML	—
后端	`StyleProfileMapper.java` + XML	—
后端	—	`WritingAgent.java` (动态风格注入)
后端	—	`ContextBuilderService.java` (构建风格上下文)
后端	—	`StreamingChapterController.java` (传风格参数)
前端	`ImportNovel.vue`	—
前端	`StyleProfilePanel.vue`	—
前端	—	`NovelDetail.vue` (集成风格面板)
前端	—	`api/index.ts` (新增API)
前端	—	`types/index.ts` (新增类型)

4.6 工作量估算

阶段	内容	时间
Phase 1	Anti-AI Prompt 优化（方案一）	1-2天
Phase 2	文风指纹表 + 基础CRUD	1天
Phase 3	导入小说功能	2天
Phase 4	文风分析引擎	2-3天
Phase 5	风格注入写作	1-2天
Phase 6	前端集成	2-3天
合计		**9-13天**

五、风险与注意事项

1. LLM 调用成本

- 文风分析需要多次调用 LLM（分析3次 + 写作时注入）
- 建议：分析结果缓存，不要每次写作都重新分析
- 导入大文本时做采样，不要全文分析

2. 风格模仿的局限性

- LLM 只能模仿"表面风格"（词汇、句式、节奏），无法模仿"深层风格"（思想深度、叙事哲学）
- 模仿效果取决于导入文本的质量和数量（建议至少5万字）
- 用户期望管理：是"风格类似"，不是"水平相当"

3. 版权问题

- 导入他人作品仅供"文风分析"，不存储/传播原文
- 生成内容必须100%原创，不得复制导入小说的情节
- 在功能说明中明确免责条款

4. 文本处理性能

- 大文件（>5MB）需要分块处理
- 分析时做章节采样，不分析全文
- 使用 MongoDB 存储导入文本，避免 MySQL 大字段问题

六、推荐实施顺序

```
立即做（今天）：Phase 1 — Anti-AI Prompt 优化
  ↓
  效果验证：生成几章对比 AI 味是否减少
  ↓
短期做（1-2周）：Phase 2-6 — 完整模仿功能
```

Phase 1 的投入产出比最高 — 改几个 prompt 模板就能显著改善文风，
不需要改架构、不需要新表、不需要前端改动。

文档版本: v1.0
创建时间: 2026-04-21
作者: Hermes Agent